

第一章 入力層～中間層

入力層に何かしらの数字を入力する。ニューラルネットワークでは重みをつけ、中間層で計算をする。

x で入力信号を送り、 w が重みを表すパラメータで各信号の重要性をコントロールする。 U が入力の総和となる。

入力： x_i

重み： w_i

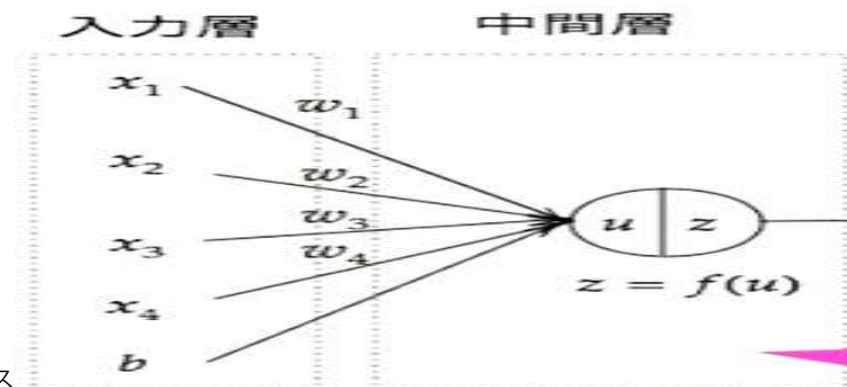
バイアス： b

総入力： u

出力： z

活性化関数： f

i : 入力層のインデックス



計算式は以下の通り

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_I \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_I \end{bmatrix} \quad .. (1.1)$$

$$u = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b \\ = Wx + b \quad .. (1.2)$$

w が傾き、 b が切片

【実践演習】

```
[7] 0 秒
✓ 秒

▶ print_vec("バイアス", b)

# 入力値
x = np.array([2, 3])
print_vec("入力", x)

# 総入力
u = np.dot(x, W) + b
print_vec("総入力", u)

# 中間層出力
z = functions.relu(u)
print_vec("中間層出力", z)

...
*** 重み ***
[[0.1]
 [0.2]]
shape: (2, 1)

*** バイアス ***
0.5
shape: ()
```

第二章 活性化関数

活性化関数・・・入力信号の総和を出力信号に変換する関数。

【事前に用意する情報】

入力: $\mathbf{x}_n = [x_{n1} \dots x_{ni}]$

訓練データ: $\mathbf{d}_n = [d_{n1} \dots d_{nk}]$

【多層ネットワークのパラメータ】

$$\mathbf{w}^{(l)} \begin{cases} \text{重み: } \mathbf{W}^{(l)} = \begin{bmatrix} w_{11}^{(l)} & \dots & w_{1l}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{j1}^{(l)} & \dots & w_{jl}^{(l)} \end{bmatrix} \\ \text{バイアス: } \mathbf{b}^{(l)} = [b_1^{(l)} \dots b_l^{(l)}] \end{cases}$$

$$\text{活性化関数: } \mathbf{f}^{(l)}(\mathbf{u}^{(l)}) = [f^{(l)}(u_1^{(l)}) \dots f^{(l)}(u_l^{(l)})]$$

$$\text{中間層出力: } \mathbf{z}^{(l)} = [z_1^{(l)} \dots z_K^{(l)}] = \mathbf{f}^{(l)}(\mathbf{u}^{(l)})$$

$$\text{総入力: } \mathbf{u}^{(l)} = \mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\text{出力: } \mathbf{y}_n^{(t)} = [y_{n1}^{(t)} \dots y_{nk}^{(t)}] = \mathbf{z}^{(L)}$$

$$\text{誤差関数: } E_n(\mathbf{w})$$

入力層ノードのインデックス: $i (= 1 \dots I)$

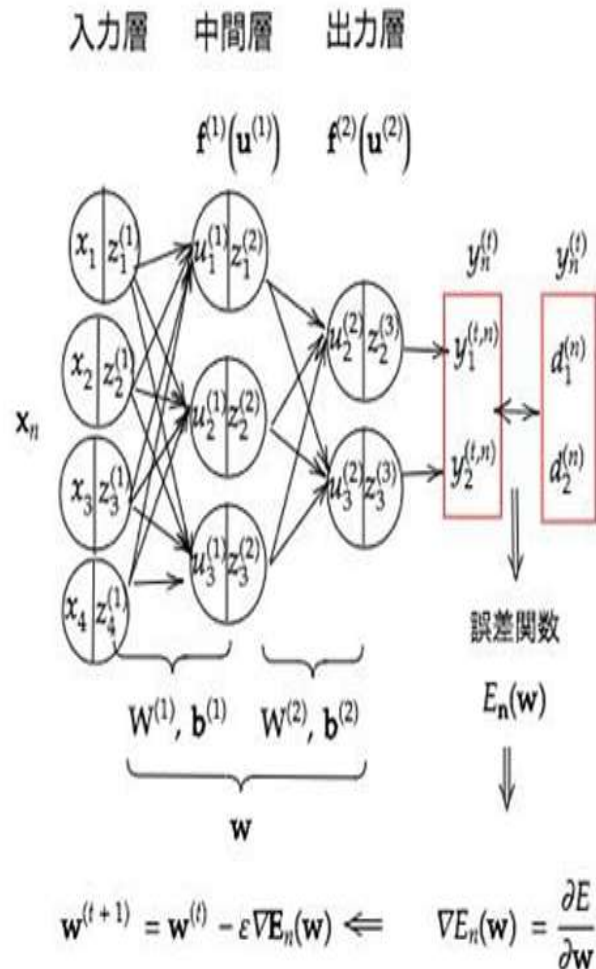
中間層ノードのインデックス: $j (= 1 \dots J)$

出力層ノードのインデックス: $k (= 1 \dots K)$

層のインデックス: $l (= 1 \dots L)$

訓練データのインデックス: $n (= 1 \dots N)$

試行回数のインデックス: $t (= 1 \dots T)$



・ 中間層用の活性化関数

ReLU関数

シグモイド（ロジスティック）関数

ステップ関数

- ・ステップ関数

しきい値を超えたら発火する関数であり、出力は常に1か0。

パーセプトロン（ニューラルネットワークの前身）で利用された関数。

課題：0 -1間の間を表現できず、線形分離可能なものしか学習できなかった

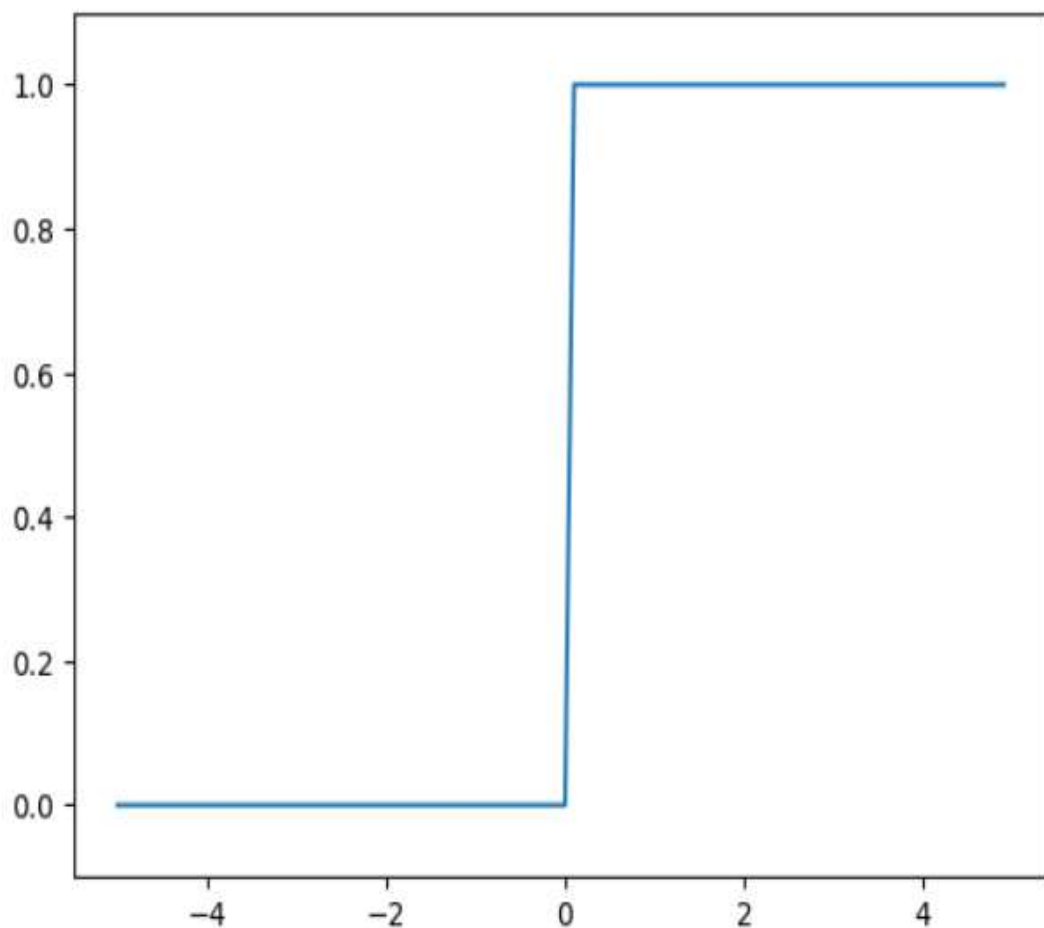
[4]
0
秒

```
import numpy as np
import matplotlib.pyplot as plt

def ste_function(x):
    return np.array(x > 0, dtype=int)

x = np.arange(-5.0, 5.0, 0.1)
y = ste_function(x)
plt.plot(x, y)
plt.ylim(-0.1, 1.1)
plt.show()
```

▼ ...



ゼロから作るDeepLearning (3.2.3 ステップ関数の実装)

・シグモイド関数（ロジスティック関数）

0 ~ 1の間を緩やかに変化する関数で、ステップ関数ではON/OFFしかない

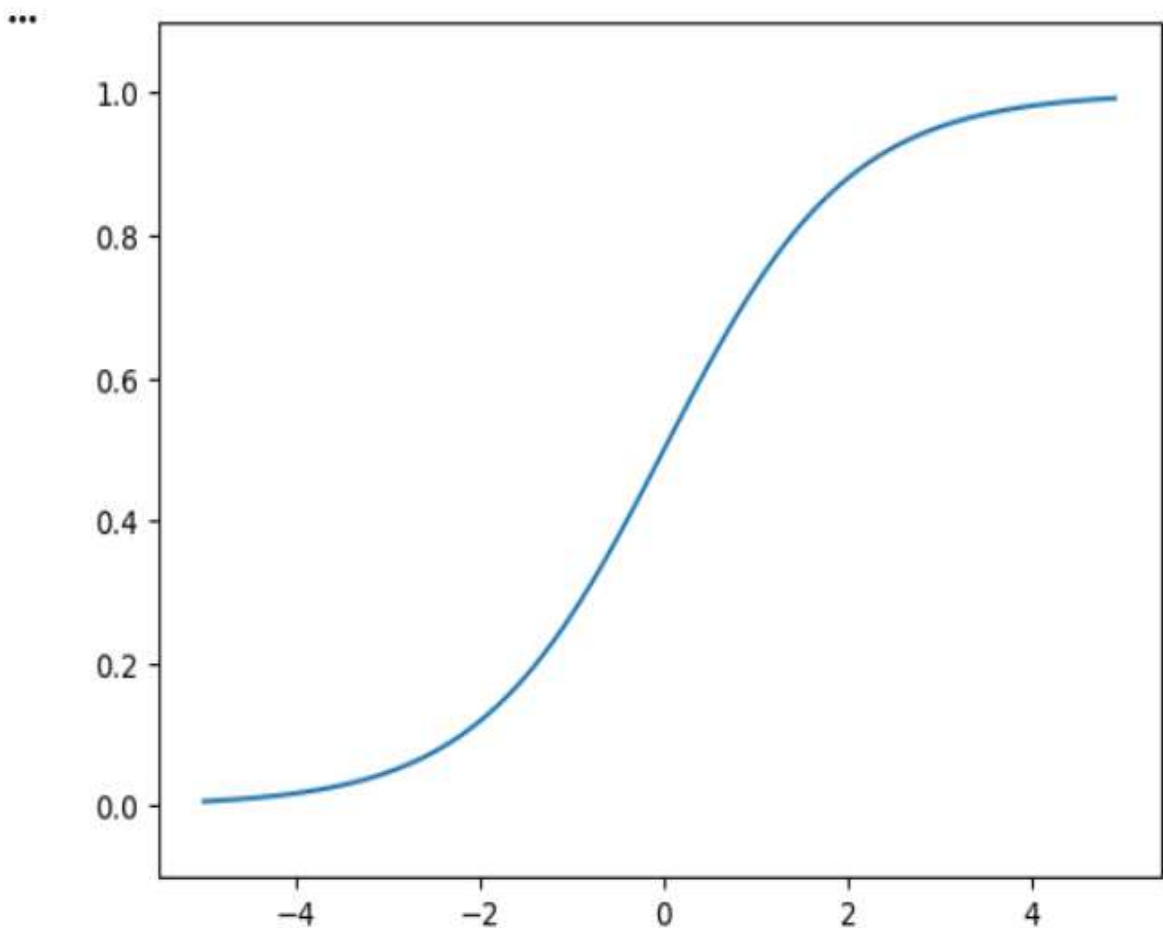
状態に対し、信号の強弱を伝えられるようになり、予想ニューラルネットワーク普及のきっかけとなった。

課題：大きな値では出力の変化が微小なため、勾配消失問題を引き起こす事があった。

[6]
0
秒

```
x= np.arange(-5.0, 5.0, 0.1)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
y = sigmoid(x)
plt.plot(x,y)
plt.ylim(-0.1,1.1)
plt.show()
```

▼



ゼロから作るDeepLearning (3.2.4 シグモイド関数の実装)

- ・ RELU関数

今最も使われている活性化関数

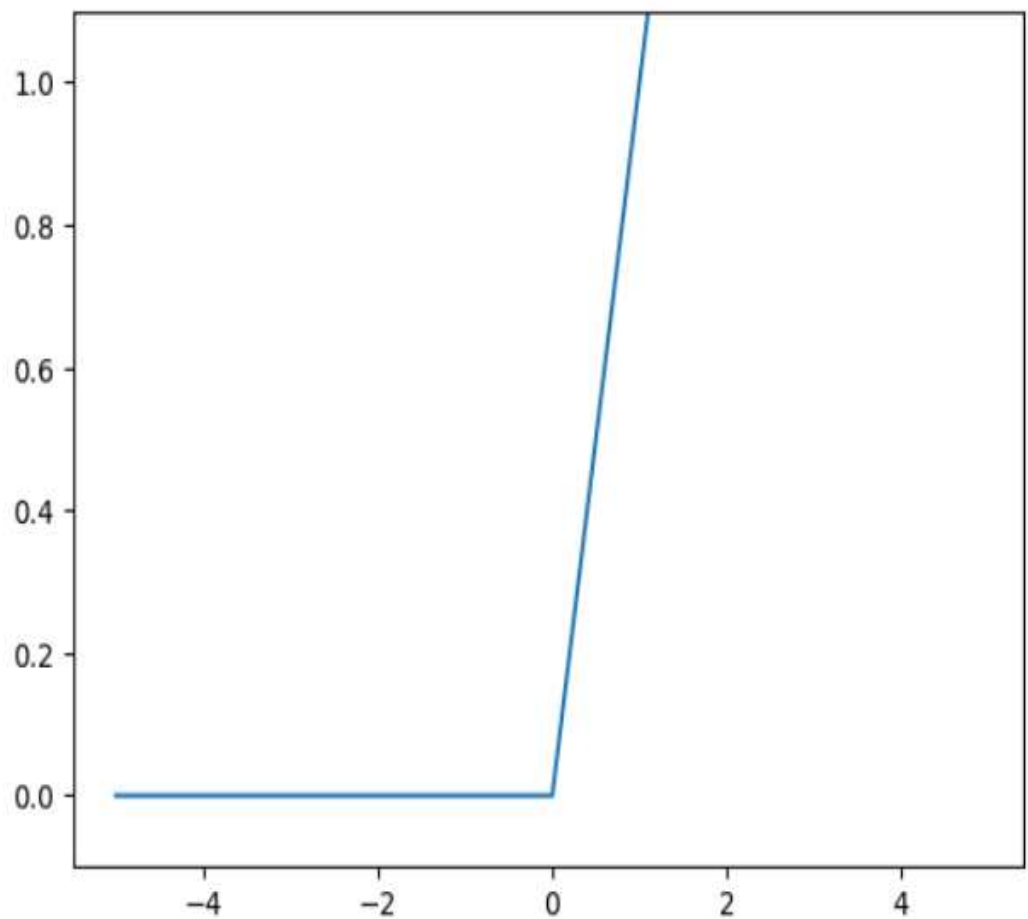
勾配消失問題の回避とスパース化に貢献することで良い成果をもたらしている。

[7]
0
秒

```
x= np.arange(-5.0,5.0,0.1)
def relu(x):
    return np.maximum(0, x)

y = relu(x)
plt.plot(x,y)
plt.ylim(-0.1,1.1)
plt.show()
```

▼



ゼロから作るDeepLearning (3.2.7 ReLU関数)

・ニューラルネットワークの行列の積

ニューラルネットワークの行列の積	
[29] ✓ 0 秒	<pre>X = np.array([1, 2]) X.shape</pre>
▼	(2,)
[30] ✓ 0 秒	<pre>W = np.array([[1, 3, 5], [2, 4, 6]]) print(W)</pre>
▼	... <pre>[[1 3 5] [2 4 6]]</pre>
[31] ✓ 0 秒	<pre>W.shape</pre>
▼	(2, 3)
[32] ✓ 0 秒	<pre>Y = np.dot(X, W) print(Y)</pre>
▼	[5 11 17]

ゼロから作るDeepLearning (3.2.3 ニューラルネットワークの行列の積)

3層ニューラルネットワークの実装

[]	<u>各層における信号伝達の実装</u>
[45] ✓ 0 秒	<pre>X = np.array([1.0, 0.5]) W1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]]) B1 = np.array([0.1, 0.2, 0.3]) print(W1.shape) print(X.shape) print(B1.shape) A1 = np.dot(X, W1) + B1</pre>
▼	... (2, 3) (2,) (3,)
[38] ✓ 0 秒	<pre>Z1 = sigmoid(A1) print(A1) print(Z1)</pre>
▼	... [0.3 0.7 1.1] [0.57444252 0.66818777 0.75026011]

[39]

✓ 0 秒

```
W2 = np.array([[0.1, 0.4], [0.2, 0.5], [0.3, 0.6]])  
B2 = np.array([0.1, 0.2])  
  
print(Z1.shape)  
print(W2.shape)  
print(B2.shape)
```

✓

```
(3, )  
(3, 2)  
(2, )
```

[40]

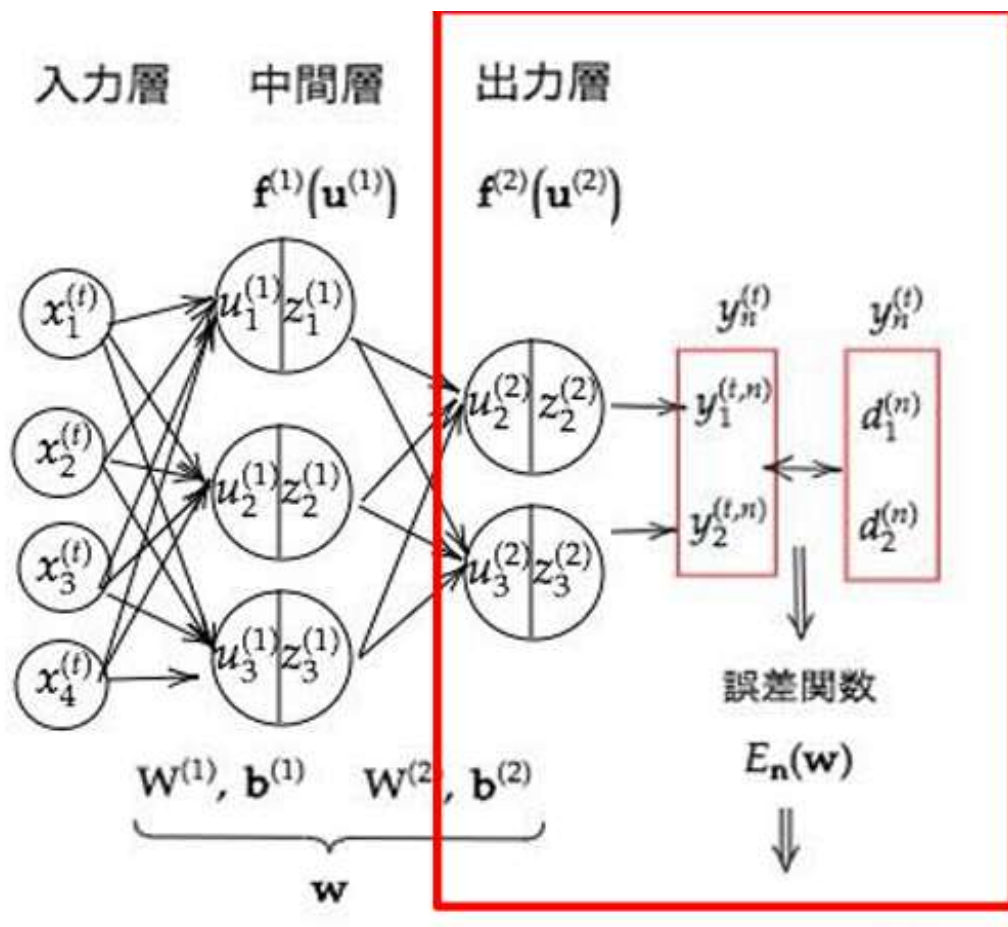
✓ 0 秒

```
A2 = np.dot(Z1, W2) + B2  
Z2 = sigmoid(A2)
```

[41]

✓ 0 秒

```
def identity_function(x):  
    return x  
  
W3 = np.array([[0.1, 0.3], [0.2, 0.4]])  
B3 = np.array([0.1, 0.2])  
  
A3 = np.dot(Z2, W3) + B3  
Y = identity_function(A3)
```



- ・ 誤差関数
- ・ 数式

$$E_n(\mathbf{w}) = \frac{1}{2} \sum_{j=1}^I (y_j - d_j)^2 = \frac{1}{2} \|\mathbf{y} - \mathbf{d}\|^2$$

- ・ コード

```
loss = functions.mean_squared_error(d, y)
```

- ・ 出力層の活性化関数
- ・ 出力層の中間層との違い

【値の強弱】

- ・ 中間層：しきい値の前後で信号の強弱を調整
- ・ 出力層：信号の大きさ（比率）はそのままに変換

【確率出力】

- ・ 分類問題の場合、出力層の出力は0 ~ 1 の範囲に限定し、総和を 1 とする必要がある

→出力層と中間層で利用される活性化関数が異なる

- ・ 中間層用の活性化関数

→ReLU関数

→シグモイド（ロジスティック）関数

→ステップ関数

- ・ 出力層用の活性化関数

→ソフトマックス関数

→恒等写像

→シグモイド関数（ロジスティック関数）

	回帰	二値分類	多クラス分類
活性化関数	恒等写像 $f(u) = u$	シグモイド関数 $f(u) = \frac{1}{1 + e^{-u}}$	ソフトマックス関数 $f(i, u) = \frac{e^{u_i}}{\sum_{k=1}^K e^{u_k}}$
誤差関数	二乗誤差	交差エントロピー	

・ソフトマックスの実装

[5]
✓ 0 秒

```
def softmax(a):  
    c = np.max(a)  
    exp_a = np.exp(a - c)  
    sum_exp_a = np.sum(exp_a)  
    y = exp_a / sum_exp_a  
  
    return y
```

[6]
✓ 0 秒

```
a = np.array([0.3, 2.9, 4.0])  
y = softmax(a)  
print(y)  
np.sum(y)
```

▼

```
[0.01821127 0.24519181 0.73659691]  
np.float64(1.0)
```

ゼロから作るDeepLearning (3.5.3 ソフトマックス関数上の実装上の注意)

・ 二乗誤差

```
[7]  
✓ 0 秒  
▶ def sum_squared_error(y, t):  
    return 0.5 * np.sum((y-t)**2)  
  
    t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]  
    y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]  
    sum_squared_error(np.array(y), np.array(t))  
  
np.float64(0.097500000000000003)
```

ゼロから作るDeepLearning (4.2.1 二乗和誤差)

・ 交差エントロピー誤差

```
[11]  
✓ 0 秒  
def cross_entropy_error(y, t):  
    delta = 1e-7  
    return -np.sum(t * np.log(y + delta))  
  
    t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]  
    y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]  
    cross_entropy_error(np.array(y), np.array(t))  
  
np.float64(0.510825457099338)
```

ゼロから作るDeepLearning (4.2.2 クロスエントロピー誤差)

第四章 勾配降下法

- ・ 勾配降下法
- ・ 深層学習の目的・・・学習を通して誤差を最小にするネットワークを作成すること
 - 誤差 $E(w)$ を最小化するパラメータ w を発見すること
 - 勾配降下法を利用してパラメータを最適化

【勾配降下法】

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \varepsilon \nabla E$$

$$\nabla E = \frac{\partial E}{\partial \mathbf{w}} = \left[\frac{\partial E}{\partial w_1} \cdots \frac{\partial E}{\partial w_M} \right]$$

- ・ 学習率
 - 学習率の値によって学習の効率が大きく異なる
 - 学習率が大きすぎた場合、最小値にいつまでもたどり着かず発散してしまう。
 - 学習率が小さい場合発散することはないが、小さすぎると収束するまでに時間がかかってしまう。
- ・ 勾配降下法の学習率の決定、収束性向上のためのアルゴリズムについて

複数の論文が公開され、よく利用されている。

 - ・ Momentum
 - ・ AdaGrad
 - ・ Adadelta
 - ・ Adam
- ・ 確率的勾配降下法 (SGD)
 - ・ 確率的勾配降下法・・・ランダムに抽出したサンプルの誤差
 - ・ 勾配降下法・・・全サンプルの平均誤差

- ・ 確率的勾配降下法のメリット
 - データが冗長な場合の計算コストの軽減
 - 望まない局所極小解に収束するリスクの軽減
 - オンライン学習ができる

```

import sys, os
sys.path.append('/content/drive/MyDrive/pythonライブラリ基礎/DNN_code_colab_day1')
from common.functions import *
from common.gradient import numerical_gradient

class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size, weight_init_std=0.01):
        self.params = {}
        self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
        self.params['b1'] = np.zeros(hidden_size)
        self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
        self.params['b2'] = np.zeros(output_size)

    def predict(self, x):
        W1, W2 = self.params['W1'], self.params['W2']
        b1, b2 = self.params['b1'], self.params['b2']

        a1 = np.dot(x, W1) + b1
        z1 = sigmoid(a1)

```

ゼロから作るDeepLearning (4.5.1 2層のニューラルネットワークのクラス)

- ・ オンライン学習

学習データが入ってくるたびに都度パラメータを更新し、学習を進めていく方法。一方、バッチ学習では一度にすべての学習データを使ってパラメータ更新を行う。

- ・ ミニバッチ勾配降下法・・・ランダムに分割したデータの集合（ミニバッチ）

D_t に属するサンプルの平均誤差

- ・ ミニバッチ勾配降下法のメリット

確率的勾配降下法のメリットを損なわず、計算機の計算資源を有効利用できる

→ CPUを利用したスレッド並列化やGPUを利用したSIMD並列化

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \varepsilon \nabla E_t$$

$$E_t = \frac{1}{N_t} \sum_{n \in D_t} E_n$$

$$N_t = |D_t|$$

- ・ 誤差勾配の計算-数値微分
- ・ 数値微分・・・プログラムで微小な数値を生成し擬似的に微分を計算する一般的な手法

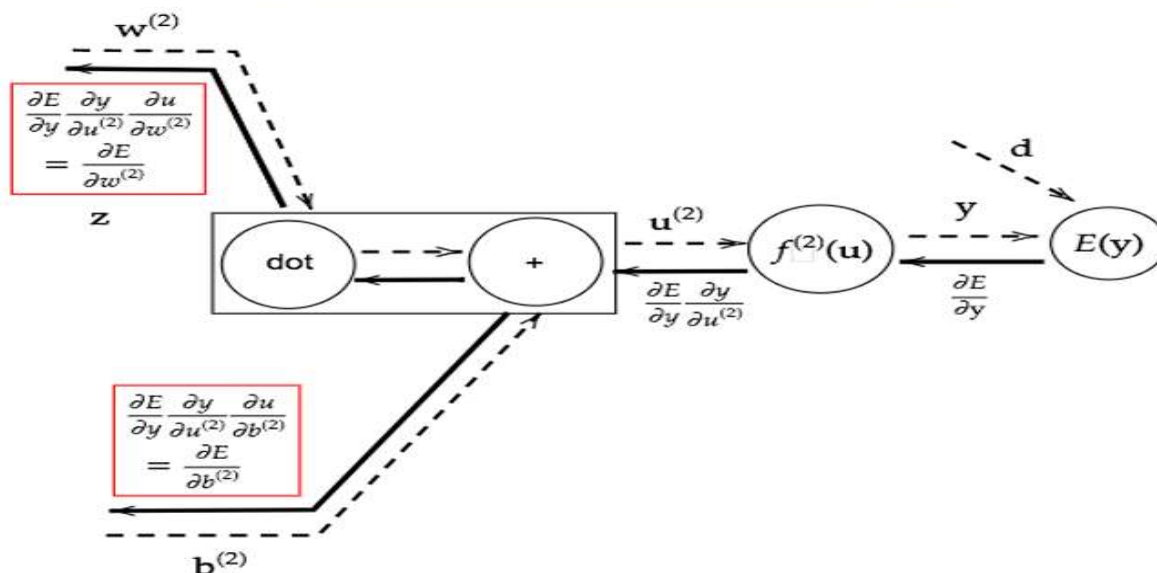
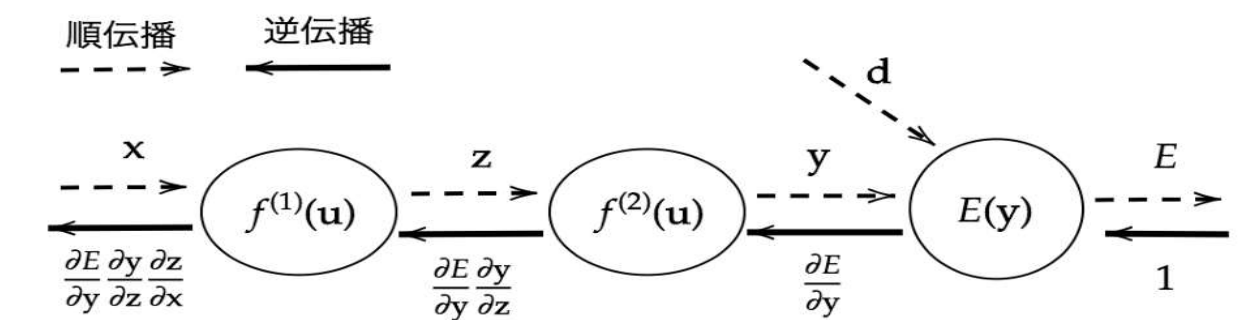
$$\frac{\partial E}{\partial w_m} \approx \frac{E(w_m + h) - E(w_m - h)}{2h}$$

- ・ 数値微分のデメリット

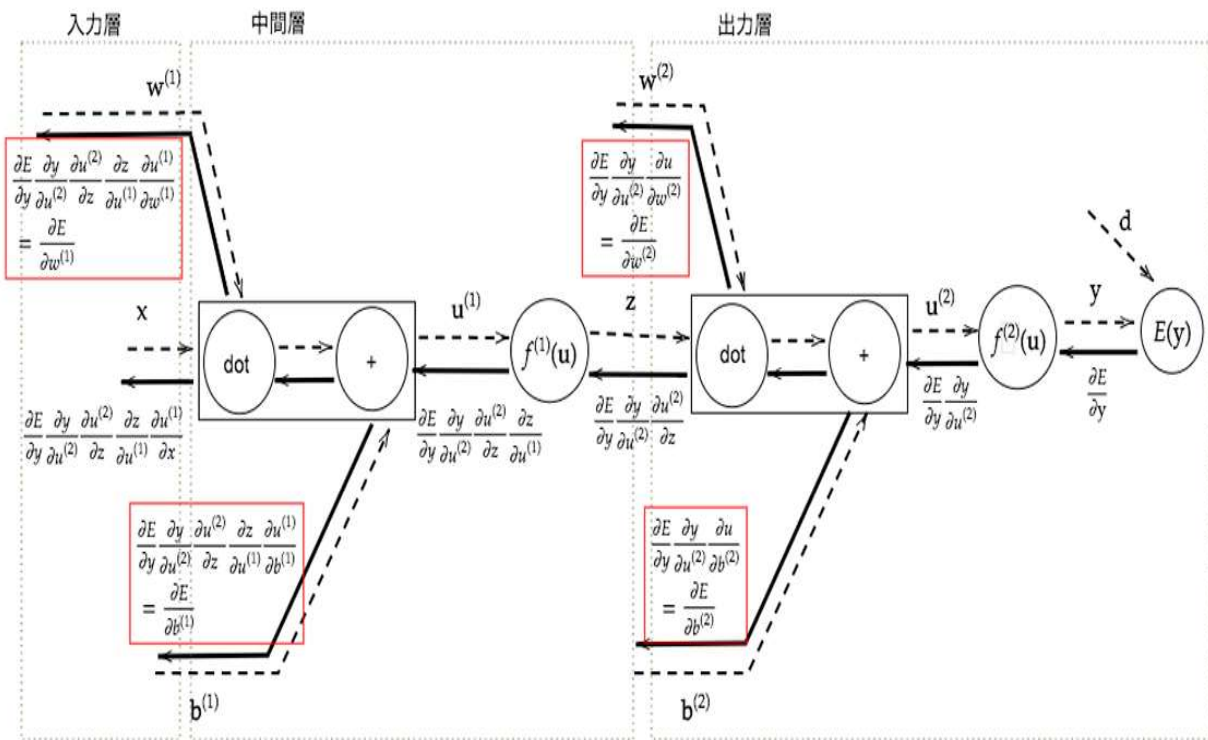
各パラメタそれぞれについて $E(W_m + h)$ や $E(W_m - 1)$ を計算するために順伝播の計算を繰り返し行う必要があり負荷が大きい
→誤差逆転伝播法を利用する。

第五章 誤差逆伝播法

誤差逆伝播法・・・算出された誤差を、出力層側から順に微分し、前の層前の層へと伝播。
最小限の計算で各パラメータでの微分値を解析的に計算する手法



計算結果 (=誤差) から微分を逆算することで、不要な再帰的計算を避けて微分を算出できる



計算結果 (=誤差) から微分を逆算することで、不要な再帰的計算を避けて微分を算出できる

▼ 準備

▼ Google Colab 用の処理

下記を実行します

- ドライブのマウント
- ノートブックファイルと同じフォルダへの移動

Googleドライブのマイドライブを基準に DNN_code/DNN_code_colab_day1 フォルダを置くことを仮定しています。必要に応じて、パスを変更してください。

```
[2]
✓ 7秒
# Google Colab での実行かを調べる
import sys
import os
ENV_COLAB = True if 'google.colab' in sys.modules else False

# google drive のマウント
if ENV_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    os.chdir('/content/drive/MyDrive/pythonライブラリ基礎/DNN_code_colab_day1/notebook')
```

▼ Mounted at /content/drive

▼ sys.pathの設定

```
[3]
✓ 0秒
import sys
sys.path.append(os.pardir) # 親ディレクトリのファイルをインポートするための設定
```

▼ importと関数定義

```
[4]
✓ 0秒
import numpy as np
from common import functions
```

```
def print_vec(text, vec):
    print("*** " + text + " ***")
    print(vec)
    print("shape: " + str(vec.shape))
    print("")
```

✓ 順伝播（単層・単ユニット）

[5]
✓ 0
秒

```
# 順伝播（単層・単ユニット）

# 重み
W = np.array([[0.1], [0.2]])

## 試してみよう 配列の初期化
#W = np.zeros(2)
#W = np.ones(2)
#W = np.random.rand(2)
#W = np.random.randint(5, size=(2))

print_vec("重み", W)

# バイアス
b = np.array(0.5)

## 試してみよう 数値の初期化
#b = np.random.rand() # 0~1のランダム数値
#b = np.random.rand() * 10 -5 # -5~5のランダム数値

print_vec("バイアス", b)

# 入力値
x = np.array([2, 3])
print_vec("入力", x)

# 総入力
u = np.dot(x, W) + b
print_vec("総入力", u)
```

```
print_vec("中間層出力", u)

# 中間層出力
z = functions.relu(u)
print_vec("中間層出力", z)
```

```
*** 重み ***
[[0.1]
 [0.2]]
shape: (2, 1)

*** バイアス ***
0.5
shape: ()

*** 入力 ***
[2 3]
shape: (2, )

*** 総入力 ***
[1.3]
shape: (1, )

*** 中間層出力 ***
[1.3]
shape: (1, )
```

順伝播（単層・複数ユニット）

[6]
0
秒

```
▶ # 順伝播（単層・複数ユニット）

# 重み
W = np.array([
    [0.1, 0.2, 0.3, 0],
    [0.2, 0.3, 0.4, 0.5],
    [0.3, 0.4, 0.5, 1],
])

## 試してみよう 配列の初期化
#W = np.zeros((4, 3))
#W = np.ones((4, 3))
#W = np.random.rand(4, 3)
#W = np.random.randint(5, size=(4, 3))
```

```
W = np.random.random((3, 4))
```

```
print_vec("重み", W)
```

```
# バイアス
```

```
b = np.array([0.1, 0.2, 0.3])
```

```
print_vec("バイアス", b)
```

```
# 入力値
```

```
x = np.array([1.0, 5.0, 2.0, -1.0])
```

```
print_vec("入力", x)
```

```
# 総入力
```

```
u = np.dot(W, x) + b
```

```
print_vec("総入力", u)
```

```
# 中間層出力
```

```
z = functions.sigmoid(u)
```

```
print_vec("中間層出力", z)
```

```
... *** 重み ***  
[[0.1 0.2 0.3 0. ]  
 [0.2 0.3 0.4 0.5]  
 [0.3 0.4 0.5 1. ]]  
shape: (3, 4)  
  
*** バイアス ***  
[0.1 0.2 0.3]  
shape: (3,)  
  
*** 入力 ***  
[ 1.  5.  2. -1.]  
shape: (4,)  
  
*** 総入力 ***  
[1.8 2.2 2.6]  
shape: (3,)  
  
*** 中間層出力 ***  
[0.85814894 0.90024951 0.93086158]  
shape: (3,)
```

✓ 順伝播 (3層・複数ユニット)

[7]
0
秒

▶ # 順伝播 (3層・複数ユニット)

ウェイトとバイアスを設定
ネットワークを作成

```
def init_network():  
    print("##### ネットワークの初期化 #####")  
    network = {}  
  
    input_layer_size = 3  
    hidden_layer_size_1=10  
    hidden_layer_size_2=5  
    output_layer_size = 4  
  
    #試してみよう  
    # 各パラメータのshapeを表示  
    # ネットワークの初期値ランダム生成  
    network['W1'] = np.random.rand(input_layer_size, hidden_layer_size_1)  
    network['W2'] = np.random.rand(hidden_layer_size_1,hidden_layer_size_2)  
    network['W3'] = np.random.rand(hidden_layer_size_2,output_layer_size)  
  
    network['b1'] = np.random.rand(hidden_layer_size_1)  
    network['b2'] = np.random.rand(hidden_layer_size_2)  
    network['b3'] = np.random.rand(output_layer_size)  
  
    print_vec("重み1", network['W1'] )  
    print_vec("重み2", network['W2'] )  
    print_vec("重み3", network['W3'] )  
    print_vec("バイアス1", network['b1'] )  
    print_vec("バイアス2", network['b2'] )  
    print_vec("バイアス3", network['b3'] )
```

```
    return network
```

プロセスを作成
x: 入力値

```
def forward(network, x):  
  
    print("##### 順伝播開始 #####")  
  
    W1, W2, W3 = network['W1'], network['W2'], network['W3']  
    b1, b2, b3 = network['b1'], network['b2'], network['b3']
```

[7]
0
秒

```

# 1層の総入力
u1 = np.dot(x, W1) + b1

# 1層の総出力
z1 = functions.relu(u1)

# 2層の総入力
u2 = np.dot(z1, W2) + b2

# 2層の総出力
z2 = functions.relu(u2)

# 出力層の総入力
u3 = np.dot(z2, W3) + b3

# 出力層の総出力
y = u3

print_vec("総入力1", u1)
print_vec("中間層出力1", z1)
print_vec("中間層出力2", z2)
print_vec("総入力2", u2)
print_vec("出力", y)
print("出力合計: " + str(np.sum(y)))

```

```

return y, z1, z2

```

```

# 入力値
x = np.array([1., 2., 4.])
print_vec("入力", x)

```

```

# ネットワークの初期化
network = init_network()

```

```

y, z1, z2 = forward(network, x)

```

▼

```

... *** 入力 ***
[1. 2. 4.]
shape: (3,)

##### ネットワークの初期化 #####
*** 重み1 ***
[[0.10061207 0.47042581 0.57105686 0.59055983 0.32303591 0.04732659

```

```

0.92462645 0.84820214 0.33058234 0.70603794]
[0.8323172 0.03563806 0.4859211 0.72820712 0.93287414 0.39729156
 0.44158297 0.61732326 0.1907347 0.47896879]
[0.4048836 0.43888484 0.23391864 0.513705 0.35654865 0.53043596
 0.72994479 0.88648402 0.32469148 0.74819369]]
shape: (3, 10)

```

*** 重み2 ***

```

[[9.79856538e-02 2.32029710e-01 7.10474245e-01 6.78815677e-01
 1.33208729e-01]
 [1.25745093e-01 5.02034117e-01 8.35303311e-01 8.54675502e-01
 3.34991566e-01]
 [4.17077139e-02 7.64630278e-01 9.66319057e-01 4.83675442e-01
 5.00741041e-01]]

```

✓ 多クラス分類 (2-3-4ネットワーク)

[8]
✓ 0 秒



```

# 多クラス分類
# 2-3-4ネットワーク

```

```

# !試してみよう_ノードの構成を 3-5-6 に変更してみよう

```

```

# ウェイトとバイアスを設定
# ネットワークを作成

```

```

def init_network():
    print("##### ネットワークの初期化 #####")

```

```

#試してみよう
# 各パラメータのshapeを表示
# ネットワークの初期値ランダム生成

```

```

network = {}

```

```

input_layer_size = 3
hidden_layer_size=50
output_layer_size = 6

```

```

#試してみよう
# 各パラメータのshapeを表示
# ネットワークの初期値ランダム生成

```

```

network['W1'] = np.random.rand(input_layer_size, hidden_layer_size)
network['W2'] = np.random.rand(hidden_layer_size, output_layer_size)

```

```

network['b1'] = np.random.rand(hidden_layer_size)

```

```

network['b1'] = np.random.rand(hidden_layer_size)
network['b2'] = np.random.rand(output_layer_size)

print_vec("重み1", network['W1'])
print_vec("重み2", network['W2'])
print_vec("バイアス1", network['b1'])
print_vec("バイアス2", network['b2'])

return network

# プロセスを作成
# x: 入力値
def forward(network, x):

    print("##### 順伝播開始 #####")
    W1, W2 = network['W1'], network['W2']
    b1, b2 = network['b1'], network['b2']

    # 1層の総入力
    u1 = np.dot(x, W1) + b1

    # 1層の総出力
    z1 = functions.relu(u1)

    # 2層の総入力
    u2 = np.dot(z1, W2) + b2

    # 出力値
    y = functions.softmax(u2)

    print_vec("総入力1", u1)
    print_vec("中間層出力1", z1)
    print_vec("総入力2", u2)
    print_vec("出力1", y)
    print("出力合計: " + str(np.sum(y)))

    return y, z1

## 事前データ
# 入力値
x = np.array([1., 2., 3.])

# 目標出力
d = np.array([0, 0, 0, 1, 0, 0])

# ウェイトとバイアスの初期化

```

```

# ネットワークの初期化
network = init_network()

# 出力
y, z1 = forward(network, x)

# 誤差
loss = functions.cross_entropy_error(d, y)

## 表示
print("\n##### 結果表示 #####")
print_vec("出力", y)
print_vec("訓練データ", d)
print_vec("交差エントロピー誤差", loss)

```

```

... ##### ネットワークの初期化 #####
*** 重み1 ***
[[0.47420061 0.90929987 0.5140314 0.60053696 0.68549534 0.00780119
 0.2324046 0.53257874 0.39387397 0.15871943 0.84422747 0.22022433
 0.500057 0.9137105 0.28890088 0.98022308 0.9730635 0.68560445
 0.20314264 0.49741166 0.50844853 0.61438156 0.66828569 0.90468773
 0.80551854 0.25659222 0.33083604 0.03637082 0.08582801 0.15966096
 0.43999795 0.63826845 0.17564717 0.23133315 0.05133845 0.47278447
 0.01513547 0.1197071 0.44785437 0.29778539 0.39106786 0.69000281
 0.61326545 0.59738107 0.18444741 0.97956138 0.74814661 0.78812517
 0.44134934 0.73380285]
[0.90645271 0.24222192 0.01103904 0.56375252 0.44038582 0.10406762
 0.32916714 0.29574532 0.78164374 0.34087344 0.70358341 0.51341697
 0.49126478 0.4900113 0.830093 0.55297265 0.19849952 0.38247076
 0.56299995 0.39174278 0.4187875 0.03085765 0.33741829 0.9760827
 0.64067839 0.81767046 0.43083502 0.1716867 0.29040103 0.43817633
 0.00726168 0.08517467 0.00131503 0.50030617 0.00277132 0.05022712]
*** 中間層出力1 ***
[4.54073936 3.49177145 4.20314148 3.40451595 2.21251891 2.1911643
 3.58812726 2.23155253 2.68230908 2.22992012 4.6647788 1.60447023
 2.43852661 2.84717155 2.87146552 5.6805186 2.97339922 3.21531574
 3.91895523 4.5941237 2.65103169 2.20078077 3.06785781 5.82066647
 3.13021829 5.58557532 1.91631474 1.42713258 1.3891285 4.39056526
 5.40176861 3.09995799 3.63749341 3.77605102 2.81316098 4.58191464
 0.84373887 3.62900648 2.61004102 2.98208138 4.740776 4.34272602
 5.10919052 2.33351195 2.84328926 4.32552788 2.46447083 3.20734147
 2.96269596 3.91115423]
shape: (50,)

*** 総入力2 ***
[79.27260901 90.22224679 69.03357185 80.06266212 89.20479136 93.48680659]
shape: (6, )

```

```
shape: (6,)
```

*** 出力 ***

```
[6.38004894e-07 3.63238004e-02 2.28069218e-11 1.40585237e-06  
1.31315502e-02 9.50542606e-01]
```

```
shape: (6,)
```

出力合計: 0.9999999999999999

結果表示

*** 出力 ***

```
[6.38004894e-07 3.63238004e-02 2.28069218e-11 1.40585237e-06  
1.31315502e-02 9.50542606e-01]
```

```
shape: (6,)
```

*** 訓練データ ***

```
[0 0 0 1 0 0]
```

```
shape: (6,)
```

*** 交差エントロピー誤差 ***

```
13.40615146047476
```

```
shape: ()
```

▽ 回帰 (2-3-2ネットワーク)

[9]
✓ 0
秒



回帰

2-3-2ネットワーク

!試してみよう_ノードの構成を 3-5-4 に変更してみよう

ウェイトとバイアスを設定

ネットワークを作成

```
def init_network():
```

```
    print("##### ネットワークの初期化 #####")
```

```
    input_layer_size = 3
```

```
    hidden_layer_size=50
```

```
    output_layer_size = 2
```

#試してみよう

各パラメータのshapeを表示

ネットワークの初期値ランダム生成

```
network['W1'] = np.random.rand(input_layer_size, hidden_layer_size)
```

```
network['W2'] = np.random.rand(hidden_layer_size, output_layer_size)
```

```

network['b1'] = np.random.rand(hidden_layer_size)
network['b2'] = np.random.rand(output_layer_size)

print_vec("重み1", network['W1'])
print_vec("重み2", network['W2'])
print_vec("バイアス1", network['b1'])
print_vec("バイアス2", network['b2'])

return network

```

プロセスを作成

```

def forward(network, x):
    print("##### 順伝播開始 #####")

    W1, W2 = network['W1'], network['W2']
    b1, b2 = network['b1'], network['b2']
    # 隠れ層の総入力
    u1 = np.dot(x, W1) + b1
    # 隠れ層の総出力
    z1 = functions.relu(u1)
    # 出力層の総入力
    u2 = np.dot(z1, W2) + b2
    # 出力層の総出力
    y = u2

    print_vec("総入力1", u1)
    print_vec("中間層出力1", z1)
    print_vec("総入力2", u2)
    print_vec("出力1", y)
    print("出力合計: " + str(np.sum(y)))

    return y, z1

```

入力値

```

x = np.array([1., 2., 3.])
network = init_network()
y, z1 = forward(network, x)
# 目標出力
d = np.array([2., 4.])
# 誤差
loss = functions.mean_squared_error(d, y)

```

表示

```

## 表示
print("\n#### 結果表示 ####")
print_vec("中間層出力", z1)
print_vec("出力", y)
print_vec("訓練データ", d)
print_vec("二乗誤差", loss)

```

ネットワークの初期化

*** 重み ***

```

[[7.76530295e-01 2.33333349e-03 9.76005628e-01 7.47181789e-01
 2.28614910e-01 6.42653188e-01 4.66931120e-01 4.67936541e-01
 2.41553940e-01 3.28551945e-01 8.84160969e-02 9.03154034e-01
 4.62321177e-01 7.49587588e-01 3.37273720e-01 1.42993134e-02
 9.05637897e-02 1.75316255e-01 3.07676683e-01 4.31658037e-01
 3.87878297e-01 1.78445544e-01 3.67669944e-01 6.24331300e-01
 3.87765412e-01 8.46202247e-01 7.20471587e-01 1.52277465e-01
 5.28408764e-01 6.95487308e-01 2.53300651e-01 2.92877486e-01
 3.46687254e-01 6.72693013e-01 9.67834051e-02 8.04050189e-01
 9.01258348e-04 7.82441010e-01 7.32906476e-01 9.59738326e-01
 8.74466687e-01 2.66908828e-01 9.98882580e-01 2.19543928e-01
 9.42905675e-01 6.43476157e-01 6.32659006e-01 2.94173021e-01
 1.47382251e-02 7.90909937e-01]
[6.01012619e-01 9.83289864e-01 2.58950714e-01 4.57191274e-02
 5.35520427e-01 6.27049978e-01 9.91414990e-01 9.96602623e-01
 7.50544056e-01 3.83109540e-01 5.94392811e-01 6.16470207e-01
 6.50857437e-01 7.29164883e-01 9.86146170e-01 9.96127103e-01
 0.54007419e-01 7.07751150e-01 2.22027020e-01 4.42545510e-01]

```

結果表示

*** 中間層出力 ***

```

[[3.01265228 5.18205155 3.76008913 2.71416822 2.97579964 3.28429544
 3.58337143 5.19900115 3.91610708 3.68975237 4.40492533 5.48565378
 5.72054547 3.96365793 4.05940292 4.70781761 4.5027373 3.34873061
 3.8747886 2.11930507 2.76739285 2.71391281 4.64288427 2.76692314
 3.74679482 3.16727787 3.90104747 4.95215652 3.32308037 2.5508077
 4.25028181 3.87136192 4.74814083 4.10826445 2.35723673 3.09649236
 3.91688459 3.75750116 2.69375138 3.76302122 5.81850252 2.98472833
 2.62241198 4.31238043 4.31845603 2.89264613 5.15218635 1.9464803
 2.22006334 5.88156778]

```

shape: (50,)

*** 出力 ***

```

[[93.09877664 94.23887201]

```

shape: (2,)

*** 訓練データ ***

```

[[2. 4.]

```

```
shape: (2,)
```

```
*** 二乗誤差 ***
```

```
4110.5102816516555
```

```
shape: ()
```

2値分類 (2-3-1ネットワーク)

[10]
✓ 0
秒

```
▶ # 2値分類
# 2-3-1ネットワーク

# !試してみよう_ノードの構成を 5-10-20-1 に変更してみよう

# ウェイトとバイアスを設定
# ネットワークを作成
def init_network():
    print("##### ネットワークの初期化 #####")

    network = {}
    network['W1'] = np.array([
        [0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1],
        [0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1],
        [0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1],
        [0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1],
        [0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1, 0.3, 0.5, 0.1]
    ])
    network['W2'] = np.random.rand(10, 20)
    network['W3'] = np.random.rand(20, 1)

    network['b1'] = np.random.rand(10)
    network['b2'] = np.random.rand(20)
    network['b3'] = np.random.rand(1)

    return network

# プロセスを作成
def forward(network, x):
    print("##### 順伝播開始 #####")

    W1, W2, W3 = network['W1'], network['W2'], network['W3']
```

```

b1, b2, b3 = network['b1'], network['b2'], network['b3']

# 隠れ層の総入力
u1 = np.dot(x, W1) + b1
# 隠れ層1の総出力
z1 = functions.relu(u1)
# 隠れ層2層への総入力
u2 = np.dot(z1, W2) + b2
# 隠れ層2の出力
z2 = functions.relu(u2)

u3 = np.dot(z2, W3) + b3
z3 = functions.sigmoid(u3)
y = z3
print_vec("総入力1", u1)
print_vec("中間層出力1", z1)
print_vec("総入力2", u2)
print_vec("出力1", y)
print("出力合計: " + str(np.sum(y)))

return y, z1

```

```

# 入力値
x = np.array([1., 2., 2., 4., 5.])

```

```

# 目標出力
d = np.array([1])

```

▶ y, z1 = forward(network, x)

```

# 誤差
loss = functions.cross_entropy_error(d, y)

```

```

## 表示
print("\n##### 結果表示 #####")
print_vec("中間層出力", z1)
print_vec("出力", y)
print_vec("訓練データ", d)
print_vec("交差エントロピー誤差", loss)

```

▼ ... ##### ネットワークの初期化 #####
順伝播開始 #####
*** 総入力 ***
[2.19970585 5.05289743 7.50775709 2.14435845 4.36783536 7.05253362
1.97373706 4.74565353 7.69607832 2.26812019]
shape: (10.)

[10]

✓ 0 秒

*** 中間層出力1 ***

[2.19970585 5.05289743 7.50775709 2.14435845 4.36783536 7.05253362
1.97373706 4.74565353 7.69607832 2.26812019]

shape: (10,)

*** 総入力2 ***

[17.37584937 24.8542248 21.84121289 25.05515679 35.55218198 23.06677956
22.72549487 17.34735215 22.46692147 24.2827842 27.62807115 19.92232795
26.77777328 24.76544982 13.2827695 21.13639592 21.18923656 15.12925672
17.75851183 25.45993516]

shape: (20,)

*** 出力1 ***

[1.]

shape: (1,)

出力合計: 1.0

結果表示

*** 中間層出力 ***

[2.19970585 5.05289743 7.50775709 2.14435845 4.36783536 7.05253362
1.97373706 4.74565353 7.69607832 2.26812019]

shape: (10,)

*** 出力 ***

[1.]

shape: (1,)

*** 訓練データ ***

[1]

shape: (1,)

*** 交差エントロピー誤差 ***

-9.999999505838704e-08

shape: ()

▽ Google Colab 用の処理

下記を実行します

- ドライブのマウント
- ノートブックファイルと同じフォルダへの移動

Googleドライブのマイドライブを基準に DNN_code/DNN_code_colab_day1 フォルダを置くことを仮定しています。必要に応じて、パスを変更してください。

```
[1] 56 秒
# Google Colab での実行かを調べる
import sys
import os
ENV_COLAB = True if 'google.colab' in sys.modules else False

# google drive のマウント
if ENV_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    os.chdir('/content/drive/MyDrive/pythonライブラリ基礎/DNN_code_colab_day1')
```

▽ ... Mounted at /content/drive

▽ sys.pathの設定

```
[2] 1 秒
from google.colab import drive
drive.mount('/content/drive')
```

▽ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ]
import sys
sys.path.append(os.pardir) # 親ディレクトリのファイルをインポートするための設定
```

▽ importと関数定義

```
[3]
import numpy as np
```

✓ 1 秒

```
import numpy as np
from common import functions
import matplotlib.pyplot as plt

def print_vec(text, vec):
    print("*** " + text + " ***")
    print(vec)
    #print("shape: " + str(x.shape))
    print("")
```

✓ 確率勾配降下法

[4]

✓ 0 秒



サンプルとする関数
#yの値を予想するAI

```
def f(x):
    y = 3 * x[0] + 2 * x[1]
    return y
```

初期設定

```
def init_network():
    # print("##### ネットワークの初期化 #####")
    network = {}
    nodesNum = 10
    network['W1'] = np.random.randn(2, nodesNum)
    network['W2'] = np.random.randn(nodesNum)
    network['b1'] = np.random.randn(nodesNum)
    network['b2'] = np.random.randn()

    # print_vec("重み1", network['W1'])
    # print_vec("重み2", network['W2'])
    # print_vec("バイアス1", network['b1'])
    # print_vec("バイアス2", network['b2'])

    return network
```

[4]

✓ 0 秒



順伝播

```
def forward(network, x):
    # print("##### 順伝播開始 #####")

    W1, W2 = network['W1'], network['W2']
    b1, b2 = network['b1'], network['b2']
```

```

u1 = np.dot(x, W1) + b1
z1 = functions.relu(u1)

## 試してみよう
#z1 = functions.sigmoid(u1)

u2 = np.dot(z1, W2) + b2
y = u2

# print_vec("総入力1", u1)
# print_vec("中間層出力1", z1)
# print_vec("総入力2", u2)
# print_vec("出力1", y)
# print("出力合計: " + str(np.sum(y)))

return z1, y

# 誤差逆伝播
def backward(x, d, z1, y):
    # print("\n##### 誤差逆伝播開始 #####")

    grad = {}

    W1, W2 = network['W1'], network['W2']
    b1, b2 = network['b1'], network['b2']

    # 出力層でのデルタ
    delta2 = functions.d_mean_squared_error(d, y)
    # b2の勾配
    grad['b2'] = np.sum(delta2, axis=0)
    # W2の勾配
    grad['W2'] = np.dot(z1.T, delta2)
    # 中間層でのデルタ
    #delta1 = np.dot(delta2, W2.T) * functions.d_relu(z1)

    ## 試してみよう
    delta1 = np.dot(delta2, W2.T) * functions.d_sigmoid(z1)

    delta1 = delta1[np.newaxis, :]
    # b1の勾配
    grad['b1'] = np.sum(delta1, axis=0)
    x = x[np.newaxis, :]
    # W1の勾配
    grad['W1'] = np.dot(x.T, delta1)

```

```

# print_vec("偏微分_重み1", grad["W1"])
# print_vec("偏微分_重み2", grad["W2"])
# print_vec("偏微分_バイアス1", grad["b1"])
# print_vec("偏微分_バイアス2", grad["b2"])

return grad

# サンプルデータを作成
data_sets_size = 100000
data_sets = [0 for i in range(data_sets_size)]

for i in range(data_sets_size):
    data_sets[i] = {}
    # ランダムな値を設定
    data_sets[i]['x'] = np.random.rand(2)

## 試してみよう_入力値の設定
# data_sets[i]['x'] = np.random.rand(2) * 10 -5 # -5~5のランダム数値

# 目標出力を設定
data_sets[i]['d'] = f(data_sets[i]['x'])

losses = []
# 学習率
learning_rate = 0.07

# 抽出数
epoch = 1000

# パラメータの初期化
network = init_network()
# データのランダム抽出
random_datasets = np.random.choice(data_sets, epoch)

# 勾配降下の繰り返し
for dataset in random_datasets:
    x, d = dataset['x'], dataset['d']
    z1, y = forward(network, x)
    grad = backward(x, d, z1, y)
    # パラメータに勾配適用
    for key in ('W1', 'W2', 'b1', 'b2'):
        network[key] -= learning_rate * grad[key]

```

```
# 誤差
loss = functions.mean_squared_error(d, y)
losses.append(loss)
```

```
print("##### 結果表示 #####")
lists = range(epoch)
```

```
plt.plot(lists, losses, '.')
# グラフの表示
plt.show()
```

... ##### 結果表示 #####

